AUTOSCRAPER: A Progressive Understanding Web Agent for Web Scraper Generation

Wenhao Huang[♢], **Zhouhong Gu**[♢], **Chenghao Peng**[♡], **Zhixu Li**[♢], **Jiaqing Liang**[♡†]
Yanghua Xiao[♢†], **Liqian Wen**[♠], **Zulong Chen**[♠]
[♢]Shanghai Key Laboratory of Data Science, School of Computer Science, Fudan University
[♡]School of Data Science, Fudan University, [♠]Alibaba Holding-Aicheng Technology-Enterprise
{whhuang21,zhgu22,chpeng23}@m.fudan.edu.cn,
{liangjiaqing,zhixuli,shawyh}@fudan.edu.cn

Abstract

Web scraping is a powerful technique that extracts data from websites, enabling automated data collection, enhancing data analysis capabilities, and minimizing manual data entry efforts. Existing methods, wrappers-based methods suffer from limited adaptability and scalability when faced with a new website, while language agents, empowered by large language models (LLMs), exhibit poor reusability in diverse web environments. In this work, we introduce the paradigm of generating web scrapers with LLMs and propose AUTOSCRAPER, a two-stage framework that can handle diverse and changing web environments more efficiently. AUTOSCRAPER leverages the hierarchical structure of HTML and similarity across different web pages for generating web scrapers. Besides, we propose a new executability metric for better measuring the performance of web scraper generation tasks. We conduct comprehensive experiments with multiple LLMs and demonstrate the effectiveness of our framework. Our work is now open-source.[1]

1 Introduction

Web scraping is a process where software automates the extraction of data from websites, typically using bots or web scrapers to gather specific information (Thapelo et al., 2021). It is important because it allows for efficient data collection and aggregation, which can be crucial for market research, competitive analysis, and real-time data monitoring.

Due to the diversity of sources and information on the internet, the construction of a web scraper requires substantial human effort. Consequently, two types of methods for automatic web information acquisition have been proposed, categorized as wrapper-based and language-agent-based (Sarkhel et al., 2023). The wrapper-based method entails complex sequences of operations within customized rule-based functions, which are designed to efficiently access and retrieve desired data from websites, which is especially beneficial for structured websites with stable layouts (Kushmerick, 1997; Dalvi et al., 2011; Bronzi et al., 2013). Conversely, the language-agent-based method leverages powerful natural language processing capabilities of large language models (LLMs) to interpret free-text queries and directly extract data within websites to meet the demand, effectively handling both structured and dynamic web content (Whitehouse et al., 2023; Marco Perini, 2024).

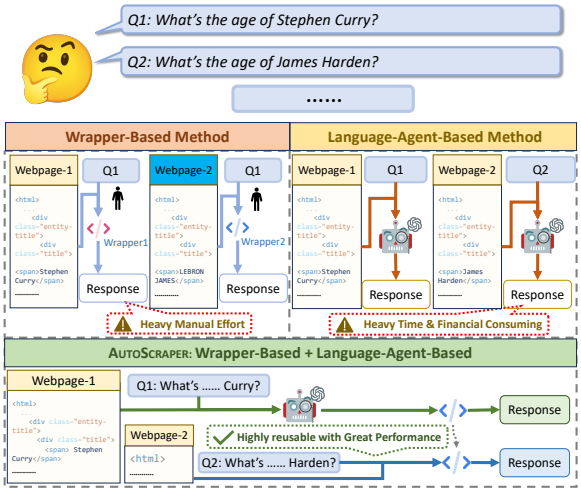


Figure 1: An illustration of comparing wrapper-based methods, language-agent-based methods and AUTOSCRAPER .

Although both types of methods facilitate web scraping to varying degrees, as shown in Figure 1, they exhibit significant shortcomings in terms of scalability. Wrapper-based method, while reusable, struggles with entirely new website structures, which necessitates extensive human effort to develop additional customized functions (Gulhane et al., 2011; Lockard et al., 2019). Conversely,

[†]Corresponding authors.
[1]Resources of this paper can be found at https://github.com/EZ-hwh/AutoScraper

Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing, pages 2371–2389
November 12-16, 2024 ©2024 Association for Computational Linguistics

although language-agent-based methods demonstrate superior performance in adapting to new content, their reliance on a limited number of superpowerful API-based LLMs for web scraping incurs considerable time and financial costs. Together, these challenges impede the broader adoption and scalability of current web scraping technologies, limiting their practicality in dynamic and diverse web environments.

To address the shortcomings of the aforementioned two paradigms, the paradigm of generating web scrapers with LLMs would be the optimal solution. On one hand, compared to wrapper-based methods, it fully leverages the reasoning and reflection capacities of LLMs, reducing manual design on new tasks and enhancing scalability. On the other hand, compared to language-agent-based methods, it introduces repeatable extraction procedures, reducing the dependency on LLMs when dealing with similar tasks, and thereby improving efficiency when handling a large number of web tasks. However, there are several challenges associated with using LLMs to generate web scrapers:

1. **Long HTML document.** Although LLMs excel in comprehending long textual content, HTML, as semi-structured data, comprises both structured (tags and attributes) and unstructured (textual content) elements. Consequently, it is challenging for LLMs to generate executable web scrapers that strictly adhere to the hierarchical structure of web pages in complex markup contexts.

2. **Reusability.** A good scraper needs to be reusable across multiple web pages. However, the differences in content and structure between various web pages can lead to the creation of a scraper that references a webpage, which can only be applied to some web pages.

3. **Appropriate evaluation metrics.** For a scraper to be considered useful, it must be able to automatically extract the desired results from all web pages. However, existing evaluation metrics for web information extraction, which focus on the extraction results from individual web pages, do not adequately reflect the usability of the scraper. This can potentially mislead experimental conclusions.

We introduce AUTOSCRAPER, a two-stage framework to address the web scraper generation task. Illustrated in Figure 2, AUTOSCRAPER comprises two main components: progressive generation and synthesis. The progressive generation stage leverages the hierarchical structure of HTML for progressive understanding to address the long HTML document. Subsequently, the synthesis module integrates multiple scrapers generated on different web pages to produce a cohesive, website-specific scraper that functions universally within that site. Besides, we propose a new evaluation metric for web scraper generation tasks, called the executability metric. Unlike traditional information extraction metrics that measure single web pages, this metric measures multiple web pages within a website, accurately reflecting the reliability and reusability of the scraper.

We evaluate AUTOSCRAPER on three available datasets with 8 LLMs. On all three datasets, AUTOSCRAPER consistently outperforms all baselines and achieves new state-of-the-art results in zero-shot settings. Also, AUTOSCRAPER can surpass supervised learning methods. Moreover, AUTOSCRAPER demonstrates superior efficiency on large-scale web information extraction tasks. Compared to traditional wrappers, AUTOSCRAPER adjusted more quickly according to different websites and task requirements. This flexibility enables scrappers to handle diverse and changing web environments more efficiently. Compared to the language agent paradigm, it introduces intermediate functions to enhance reusability and reduce the dependency on LLMs when dealing with similar tasks, thereby improving efficiency when handling a large number of web tasks.

2 Related Work

Wrapper-based methods for web scraping utilize the hierarchical structure of the webpage. Method of this category includes rule-based (Zheng et al., 2008), learning wrappers (i.e a DOM-specific parser that can extract content) (Gulhane et al., 2011; Kushmerick, 1997; Dalvi et al., 2011), heuristic algorithm (Lockard et al., 2018, 2019) and deep learning neural network (Lin et al., 2020; Zhou et al., 2021; Li et al., 2022; Wang et al., 2022). These methods demand substantial human involvement, including creating wrapper annotations, applying heuristic scoring rules (such as visual proximity), crafting features for neural network input, and using prior knowledge for verification. Therefore, it is difficult for wrapper-based methods to

automatically scale up when facing web scraping tasks across a large number of different websites.

With the emergence of powerful LLMs (OpenAI, 2023; Touvron et al., 2023), language agents (Sumers et al., 2023) now operate in interactive environments, leveraging LLM-based reasoning, grounding, learning, and decision-making. General language agents, such as Chain-of-Thought (Wei et al., 2023), Reflexion (Shinn et al., 2023), Self-Refine (Madaan et al., 2023), and Self-Debug (Chen et al., 2023), capitalize on LLMs' self-reflection capabilities for iterative planning optimization. However, these agents do not effectively utilize web structural features and fail to simplify the web environment after unsuccessful planning attempts, limiting the optimization of subsequent planning.

Current language agents primarily aim to streamline the web environment (Sridhar et al., 2023; Gur et al., 2023; Zheng et al., 2024) and develop strategies for planning and interacting with the web (Sodhi et al., 2023; Ma et al., 2023). Nevertheless, these frameworks mainly focus on the concept of open-world web simulation environments (Shi et al., 2017; Yao et al., 2023; Deng et al., 2023; Zhou et al., 2023), which encompass a broad spectrum of tasks found in real-life scenarios, such as online shopping, flight booking, and software development. These task scenarios are oriented towards individuals and have significantly different requirements for accuracy and efficiency compared to web scraping.

As a result, current language-agent-based methods cannot effectively exploit the HTML structural similarities across multiple web pages, reducing their dependency on LLMs when performing repetitive operations and leading to inefficiencies.

3 Preliminaries

In this section, we first define the scraper generation task and then present the dataset collection process and its corresponding evaluation metrics.

3.1 Task Formulation

First, we formulate our scraper generation task. Given a set of webpages on the same website $w \in \mathcal{W}$ describing a subject entity s (also called topic entity in the previous literature), and its corresponding predefined target attribute $r \in \mathcal{R}$, the task objective is to generate an executable rule/action sequence $\mathcal{A}$ to extract target information o from all webpages.

Dataset	Num_{Case}	Num_{Task}	Num_{Web}
SWDE	320	32	32,000
EXTENDED SWDE	294	221	29,400
DS1	83	11	186

Table 1: The statistic of web scraping task benchmarks. We report the number of the case ($\textbf{Num}_{\textbf{Case}}$), the number of the different extraction task ($\textbf{Num}_{\textbf{Task}}$) and the total number of webpages ($\textbf{Num}_{\textbf{Web}}$).

3.2 Datasets

We adopt the semi-structure information extraction task as a testbed for the scraper generation task.

SWDE (Hao et al., 2011) is a Structured Web Data Extraction dataset that contains webpages from 80 websites in 8 domains, with 124,291 webpages. Each of the websites from the same domains focuses on 3-5 attributes in the web pages.

EXTENDED SWDE (Lockard et al., 2019) involves fine-grained manual annotation of 21 sites in 3 domains from SWDE. While SWDE contains an average of 4,480 triples for 3 predicates per website, the EXTENDED SWDE dataset averages 41K triples for 36 predicates per site.

DS1 (Omari et al., 2017) contains 166 annotated webpages from 30 real-life large-scale websites categorized into books, shopping, hotels, and movies.

We transform the dataset with the following settings. First, we design instructions for each of the domains, and for each of the attributes as the input information for LLMs[2]. Second, for each website in each domain, we sample 100 web pages as the whole test set. We consider the set of webpages on the same websites and the corresponding extraction instruction as a case. For example, for the ESPN websites[3] in NBA player domains, the sampled 100-detail webpage of players and the instruction *Please extract the team of the player he plays now* is a complete case of our scraper generation task. Third, we pre-process the web pages by removing irrelevant elements in a webpage. We use open-source BeautifulSoup library[4] and filter out all DOM element nodes with `<script>` and `<style>`, as well as delete all attributes in the element node except `@class`. We replace the original escape characters in the annotations to ensure

[2]Further details about the prompt is in Appendix D.1

[3]https://global.espn.com/nba/

[4]https://beautifulsoup.readthedocs.io

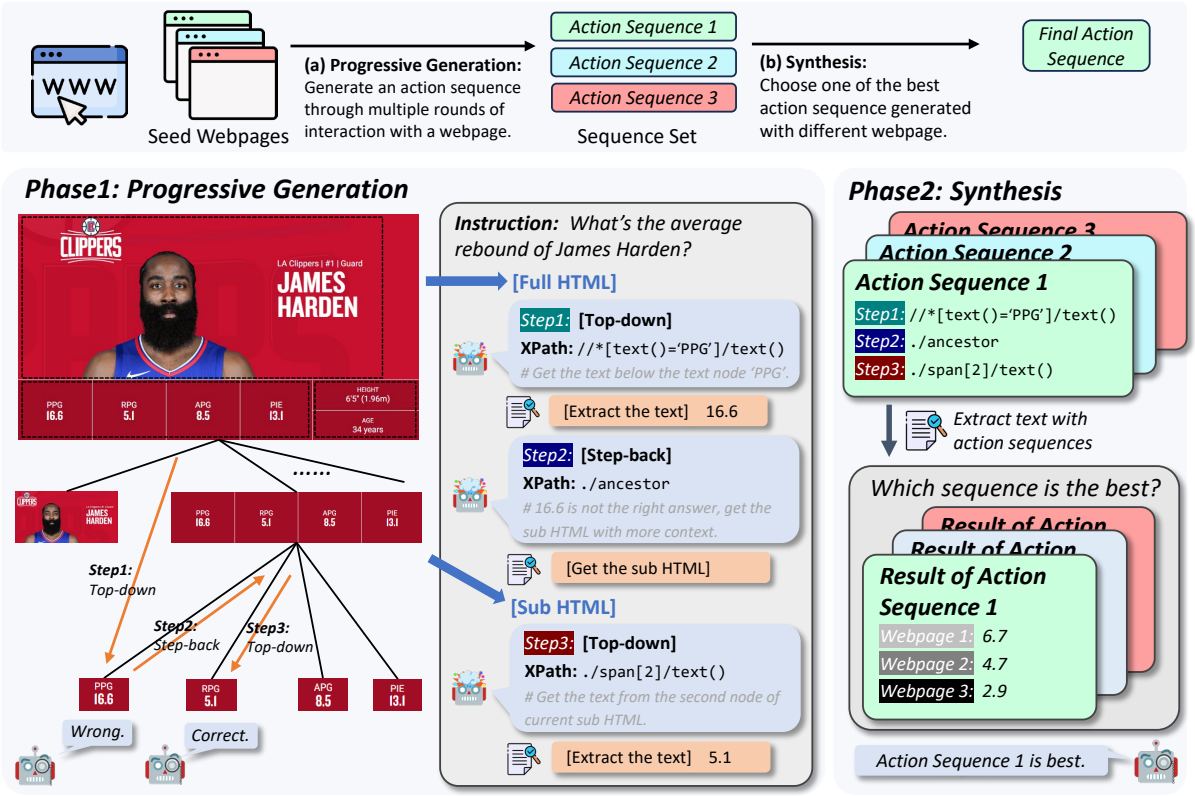


Figure 2: AUTOSCRAPER framework of two phases: **(a) progressive generation** and **(b) synthesis**.

consistency with the corresponding information on the web. The statistic of the dataset we transformed is shown in Table 1.

3.3 Evaluation Metrics

Existing evaluation schemes for web page information extraction tasks still follow the traditional metrics of text information extraction tasks, namely precision, recall, and F1 score. They limit the assessment of methods for the scraper generation task to two aspects. First, it focuses on extraction with a single webpage, rather than considering the generalizability from the perspective of a collection of webpages. Second, it does not effectively measure the transferability when adopting the action sequence to other web pages.

To address this issue, we transform the traditional IE task evaluation into an executable evaluation. Based on the traditional IE evaluation on a collection of web pages, we categorize the executability of action sequences into the following six situations. Specifically, for each extraction task on a website, the result is classified based on the extraction result on precision, recall, and f1-score. (1) **Correct**: both precision, recall and f1-score equal 1, which indicates the action sequence is precisely; (2) **Precision(Prec.)**: only precision equals 1, which indicates perfect accuracy in the instances extracted following the action sequence, but misses relevant instances; (3) **Recall(Reca.)**: only recall equals 1, which means that it successfully identifies all relevant instances in the webpage but incorrectly identifies some irrelevant instances; (4) **Un-executable(Unex.)**: recall equals 0, which indicates that the action sequence fails to identify relevant instances; (5) **Over-estimate(Over.)**: precision equals 0, which indicates that the action sequence extracts the instances while ground truth is empty; (6) **Else**: the rest of the situation, including partially extracting the information, etc.

Since the above classifications are mutually exclusive, we use the ratio metric to calculate the proportion of each result in our task.

$$M_R = \frac{\#\text{ case of situation}}{\#\text{ total case}} \quad (1)$$

We are more concerned with success rate, so for the *Correct* metric, higher values indicate a better proportion of generated execution paths; whereas for the *Un-executable* metric, lower values are preferable.

4 AUTOSCRAPER

In this section, we describe our framework AUTOSCRAPER for generating a scraper to extract specific information from semi-structured HTML. Our approach is divided into two phases: first, we adopt a progressive generation module that utilizes the hierarchical structure of web pages; second, we employ a synthesis module based on results from multiple web pages. The overall framework is presented in Figure 2.

4.1 Modeling

Unlike the wrapper method that generates an XPath, we model the scraper generation task as an action sequence generation task. In specific, we generate an action sequence $\mathcal{A}_{seq}$ that consists of a sequence of XPath[5] expression from a set of seed webpages (i.e., a small portion of webpages in the test case for generating the sequence).

$$\mathcal{A}_{seq} = [\text{XPath}_1, \text{XPath}_2, ..., \text{XPath}_n] \quad (2)$$

where n denotes the length of the action sequence. We execute the XPath in the sequence using the parser in order. In the sequence, all XPath expressions except the last one are used for pruning the web page, and the last one is used for extracting the corresponding element value from the pruned web page.

4.2 Progressive Generation

Dealing with the lengthy content and hierarchical structure of webpages, generating a complete and executable scraper in one turn is difficult. However, the HTML content is organized in a DOM tree structure, which makes it possible to prune irrelevant page components and hence, limit the length and height of the DOM tree to improve the performance of LLM generation.

Specifically, we perform a traversal strategy consisting of **top-down** and **step-back** operations. **Top-down** refers to starting from the root node of the current DOM tree, progressively refining down to the specific node containing the target information. **Step-back** refers to reassessing and adjusting selection criteria by moving up the DOM tree to choose a more reliable and broadly applicable node as a foundation for more consistent and accurate XPath targeting. At each step, we first employ a top-down operation, guiding the LLMs to directly write out the XPath leading to the node containing the target information and to judge whether the value extracted with XPath is consistent with the value it recognizes. If execution fails, then adopt a step-back operation to retreat from the failed node, ensuring the web page includes the target information, which is driven by LLMs. The detail is shown in Algorithm 1.

[5] https://en.wikipedia.org/wiki/XPath

4.3 Synthesis

Although we gain an executable action sequence within the progressive generation process, there are still differences in the specific location of the target information and the structure between different web pages. The action sequence may collect XPath with specific characteristics in a single HTML and lose generalizability. To enhance the reusability of the action sequence, we propose a synthesis phase.

Specifically, we randomly select n_s webpages from the case as seed webpages. Then, we generate an action sequence for each of them. Subsequently, we execute multiple different action sequences to extract information from the seed web pages, respectively. We collect all action sequences and their corresponding results and then choose one that can extract all the target information in the web pages as the final action sequence.

5 Experiment

Intending to put AUTOSCRAPER to practical use, we investigate the following research questions: 1) Can AUTOSCRAPER outperform the state-of-the-art scraper generation methods? 2) How does AUTOSCRAPER framework improve the performance of the scraper generation task? 3) Does AUTOSCRAPER meet the requirements for web scraping tasks, specifically being accurate and efficient?

5.1 Experimental Settings & Evaluation Metrics

We conduct our experiment on 8 LLMs including closed-source LLMs: **GPT-3.5-Turbo** (OpenAI, 2022), **Gemini Pro** (Team et al., 2023), **GPT-4-o-mini** (OpenAI, 2024) and **GPT-4-Turbo** (OpenAI, 2023) as well as open-source LLMs: **Phi-3-medium** (Abdin et al., 2024), **CodeLlama-34B** (Rozière et al., 2024), **Mixtral 8×7B** (Jiang et al., 2024) and **Deepseek-Coder-33B** (Guo et al., 2024). Furthermore, we apply different LLM-prompt-based web agents as our baselines, including **COT** (Wei et al., 2023) and **Reflexion** (Shinn

Dataset		SWDE									EXTENDED SWDE			DS1		
Models	Method	EXECUTABLE EVALUATION						IE EVALUATION			EXEC EVAL		IE EVAL	EXEC EVAL		IE EVAL
		Correct(↑)	Prec	Reca	Unex.(↓)	Over.	Else	Prec	Reca	F1	Correct	Unex.	F1	Correct	Unex.	F1
Closed-source LLMs																
GPT-3.5-Turbo	COT	36.75	8.83	6.71	43.46	0.71	3.53	89.45	50.43	47.99	35.19	55.40	41.28	32.65	53.06	41.16
	Reflexion	46.29	11.66	2.83	37.10	0.71	1.41	**94.67**	55.85	55.10	43.90	49.13	48.66	36.73	51.02	43.75
	AUTOSCRAPER	**54.84**	11.83	8.96	**19.35**	1.08	3.94	85.85	**73.34**	**69.20**	**46.34**	**34.84**	**57.74**	**48.98**	**44.90**	**52.38**
Gemini Pro	COT	29.69	10.94	7.50	47.19	1.25	3.44	81.21	45.22	41.81	34.49	49.13	42.40	17.72	75.95	22.10
	Reflexion	33.12	6.56	4.06	52.50	0.63	3.12	**87.45**	42.75	40.88	34.15	51.57	41.66	20.25	65.82	27.66
	AUTOSCRAPER	**42.81**	11.87	4.69	**34.38**	1.25	5.00	85.70	**57.54**	**54.91**	**35.89**	**42.86**	**47.80**	**43.04**	**34.18**	**56.92**
GPT-4-o-mini	COT	54.66	13.50	6.43	20.26	0.96	4.18	89.74	72.87	69.92	45.79	38.72	56.32	46.99	42.17	53.77
	Reflexion	53.70	15.11	3.22	22.83	0.96	4.18	**92.14**	70.20	69.15	39.06	47.47	48.66	38.55	45.78	43.86
	AUTOSCRAPER	**62.06**	14.15	3.86	**15.11**	0.96	3.86	91.76	**78.10**	**76.97**	**56.23**	**27.27**	**67.56**	**53.01**	**34.94**	**60.10**
GPT-4-Turbo	COT	61.88	12.50	7.19	14.37	0.94	3.12	87.75	79.90	76.95	56.10	29.27	65.08	50.60	30.12	64.73
	Reflexion	67.50	13.75	4.37	10.94	0.94	2.50	**93.28**	82.76	82.40	**64.81**	19.51	75.85	50.60	33.73	63.50
	AUTOSCRAPER	**71.56**	14.06	5.31	**4.06**	0.63	4.37	92.49	**89.13**	**88.69**	64.11	**15.33**	**76.21**	**57.83**	**16.87**	**75.52**
Open-source LLMs																
Phi-3-medium	COT	12.50	2.81	3.12	80.00	0.00	1.56	**94.38**	18.10	17.21	11.78	79.46	16.28	9.64	85.54	12.28
	Reflexion	12.19	6.56	1.87	77.81	0.00	1.56	92.45	18.21	17.31	12.66	82.28	15.42	7.23	90.36	8.89
	AUTOSCRAPER	**24.06**	12.50	7.50	**52.19**	0.31	3.44	85.07	**38.59**	**34.93**	**21.15**	**64.42**	**30.29**	**22.89**	**69.88**	**26.60**
CodeLlama	COT	17.98	3.75	2.25	74.53	0.00	1.50	**79.75**	21.98	21.36	9.01	85.84	11.21	2.70	89.19	9.19
	Reflexion	18.08	4.80	2.95	73.06	0.00	1.11	78.96	23.26	22.44	**13.73**	**80.26**	**16.01**	8.82	85.29	12.69
	AUTOSCRAPER	**23.99**	8.12	1.48	**64.94**	0.00	1.48	78.59	**28.70**	**28.41**	11.16	85.84	12.52	**13.51**	**81.08**	**17.39**
Mixtral 8×7B	COT	28.75	8.13	4.37	57.81	0.31	0.63	**89.79**	38.23	37.26	32.40	57.14	38.30	17.72	74.68	22.01
	Reflexion	36.25	6.88	3.12	51.25	0.00	2.50	89.35	44.57	43.60	29.62	62.02	33.64	22.78	69.62	28.20
	AUTOSCRAPER	**46.88**	10.62	7.19	**30.31**	0.63	4.37	87.32	**62.71**	**59.75**	**40.77**	**38.33**	**52.50**	**36.71**	**43.04**	**48.23**
Deepseek-coder	COT	36.56	10.94	5.63	42.50	0.63	3.75	86.05	48.78	47.05	**38.33**	**47.74**	**44.80**	25.30	60.24	35.65
	Reflexion	37.19	11.25	4.06	44.69	1.25	1.56	**86.41**	48.28	47.08	36.24	51.92	43.64	22.89	65.06	32.04
	AUTOSCRAPER	**38.75**	11.25	5.31	**39.69**	0.63	4.37	84.91	**52.11**	**49.68**	37.63	50.52	44.33	**39.76**	**42.17**	**50.28**

Table 2: The executable evaluation and IE evaluation of LLMs with three frameworks in SWDE, EXTENDED SWDE, and DS1 dataset. Best Correct, Unexecutable, precision, recall, and F1 score are marked **bold**.

et al., 2023) and **AUTOSCRAPER** to them. The comparison between them is discussed in Appendix B.1. Due to the limited-length context of LLMs, all experiments are conducted under zero-shot settings.

We test them on three datasets: SWDE (Hao et al., 2011), EXTEND SWDE (Lockard et al., 2019) and DS1 (Omari et al., 2017). The detailed experimental results of the last two can be found in Appendix A.1 and A.2. We set the size of seed webpages $n_s = 3$ for SWDE and EXTEND SWDE, $n_s = 1$ for DS1 and max retry times $d_{max} = 5$.

In addition to the execution evaluation metrics described in Section 3.3, we also employ traditional evaluation metrics to more comprehensively assess the quality of different action sequences. Specifically, we adopt precision (P.), recall (R.), and macro-f1 (F1), which are calculated as the mean of the corresponding metrics for each case. Detailed experimental results on the last two datasets can be found in Table 16 and 17.

5.2 Main Results

Results in Table 2 show that: 1) With AUTOSCRAPER generating action sequence, LLMs can achieve better performance. Compared to the COT and Reflexion baseline, our method performs a higher ratio of correct and a lower ratio of unexecutable. Also, it should be noted that Mixtral 8×7B + AUTOSCRAPER can outperform GPT-3.5-Turbo + Reflexion, indicating the superiority of AUTOSCRAPER in the generation of executable action sequences in the scraper generation task. 2) Models with small parameter sizes have significant difficulties in understanding and writing executable paths, so they can be considered challenging to apply in this task. On the contrary, large-scale models demonstrate a more stable ability in instruction alignment, web structure comprehension, and reflection on execution results. 3) Traditional IE evaluation metrics cannot well describe the success rate of our task. Especially for the precision metric, it fails to reveal the performance gap among different methods with different models. This is because the extraction metrics only evaluate the results that have been extracted, ignoring that unexecutable or empty extractions also greatly damage the executability.

5.3 Ablation Study

To further justify the effectiveness of each component of AUTOSCRAPER, we perform an ablation study. The results are shown in Table 3. It shows that: 1) AUTOSCRAPER without a second

Models	Method	Exec Eval		IE Eval
		Correct(↑)	Unex.(↓)	F1
GPT-3.5-Turbo	COT	36.75	43.46	47.99
	- *synthesis*	27.56	57.24	34.44
	Reflexion	46.29	37.10	55.10
	- *synthesis*	28.62	59.01	35.01
	AutoScraper	54.84	19.35	69.20
	- *synthesis*	44.52	29.33	58.44
Gemini Pro	COT	29.69	47.19	41.81
	- *synthesis*	27.56	57.24	33.09
	Reflexion	33.12	52.50	40.88
	- *synthesis*	28.62	59.01	37.60
	AutoScraper	42.81	34.38	54.91
	- *synthesis*	39.46	31.56	56.48
GPT-4-Turbo	COT	61.88	14.37	76.95
	- *synthesis*	46.88	30.00	61.20
	Reflexion	67.50	10.94	82.40
	- *synthesis*	56.87	25.31	69.78
	AutoScraper	71.56	4.06	88.69
	- *synthesis*	65.31	11.87	80.41

Table 3: Ablation study on AutoScraper. We report **Correct**, **Unexecutable** from the executive evaluation, and **F1** score from the IE evaluation in SWDE dataset.

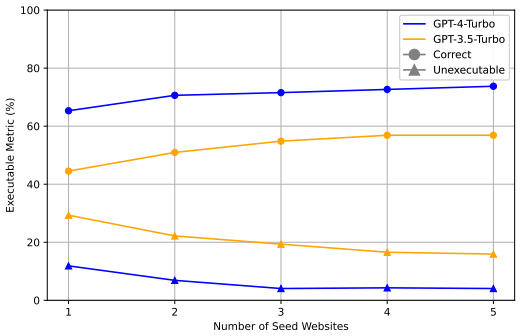


Figure 3: The performance of AutoScraper with different number of seed websites in SWDE dataset.

module still beat the other two baseline methods among different LLMs. 2) The second module of AutoScraper, **synthesis** module, not only improves AutoScraper, but also improves the performance of other methods. Using more web pages for inference can make the generated scraper more stable and have better generalization.

5.4 Seed Websites

In all previous experiments, we fixed the number of seed websites $n_s = 3$, which demonstrates the effectiveness of the synthesis module. In this experiment, we offer different numbers of seed webpages and test the performance of AutoScraper. The result is shown in Figure 3.

As the number of seed webpages increases, the correct ratio increases, while the unexecutable ratio decreases. It suggests that the performance of AutoScraper can still be further improved by providing more seed webpages. In addition, the performance improvement reduces as the number increases, which shows that there is an upper limit to improve the performance of AutoScraper by increasing the number of seed webpages.

Model	Direct Extraction	AutoScraper
GPT-3.5-Turbo	75.76	69.20
Gemini Pro	76.62	54.91
GPT-4-o-mini	79.93	76.97
GPT-4-Turbo	78.56	88.69
Phi-3-medium	71.73	34.93
Codellama	47.38	28.41
Mixtral 8×7B	73.45	59.75
Deepseek-coder	61.96	49.68

Table 4: Comparing LLM direct extraction with AutoScraper on the SWDE dataset.

6 Discussion

In this section, we will discuss other aspects of AutoScraper, including its comparison with existing website information extraction methods, efficiency analysis of AutoScraper, and the limitations of the current approach.

6.1 Comparison with LLM Direct Extraction

Since LLMs can understand human instructions and webpage text, a natural web information extraction solution involves using prompts to guide LLMs to extract target content, which we refer to as direct extraction. We compare direct extraction with AutoScraper both in zero-shot settings using each of the LLMs mentioned above.

Table 4 shows that in the direct extraction setting, the extraction performance of all LLMs other than GPT-4-Turbo is superior to that of AutoScraper. However, as the capability of LLMs improves, the gap between the two settings narrows. This indicates that: 1. While LLMs like Phi-3-medium can understand webpage content well *(i.e., correctly extract the expected content)*, they still struggle to comprehend webpage structures *(i.e., generating XPath using features like DOM tree)*. 2. AutoScraper, combined with the best current LLMs, already achieves superior extraction performance, and the framework is expected to deliver even better and more stable performance as LLMs continue to improve.

Model	F1
Render-Full (Hao et al., 2011)	84.30
FreeDOM (Lin et al., 2020)	82.32
SimpDOM (Zhou et al., 2021)	83.06
MarkupLM$_{\text{BASE}}$ (Li et al., 2022)	84.31
WebFormer (Wang et al., 2022)	86.58
Reflexion + GPT-4-Turbo	82.40
AUTOSCRAPER + GPT-4-Turbo	**88.69**

Table 5: Comparing the extraction performance (F1) of 5 baseline models to our method AUTOSCRAPER using GPT-4-Turbo on the SWDE dataset. Each value of the supervised model in the table is trained on 1 seed site.

Websites	T_d	$n_sT_g + T_s$	T_e	$N_{\mathcal{W}}$
Auto	8.27s	238.4s	0.30s	30
Book	10.20s	176.4s	0.51s	18
Camera	6.59s	107.1s	0.31s	18
Job	7.42s	123.5s	0.21s	18
Movie	7.47s	133.2s	0.21s	19
Nbaplayer	8.32s	179.4s	0.45s	23
Restaurant	8.87s	160.8s	0.54s	20
University	14.26s	134.7s	0.32s	10

Table 6: Time efficiency analysis on GPT-4-Turbo.

6.2 Comparison with supervised baselines

To further demonstrate that AUTOSCRAPER is adaptive to different web information extraction tasks, we conduct a comparison with 5 baseline models in web information extraction on supervised learning scenarios: Render-Full (Hao et al., 2011) proposes a complicated heuristic algorithm for computing visual distances between predicted value nodes and adjusting the predictions. FreeDOM (Lin et al., 2020) and SimpDOM (Zhou et al., 2021) encode textual features of DOM tree node with LSTM, while MarkupLM (Li et al., 2022) is pre-trained on HTML with text and markup information jointly. WebFormer (Wang et al., 2022) leverages the web layout for effective attention weight computation. These models are trained on webpages in some seed websites and tested on the other websites.

Table 5 shows the result. Although the comparison is unfair because our method is in zero-shot settings, AUTOSCRAPER beat all of them on F1 scores. It shows that by designing an appropriate framework, LLMs can surpass supervised learning methods in some web information extraction tasks.

6.3 Efficiency Analysis

Suppose the number of seed webpages is n_s, the number of webpages on the same website is $N_{\mathcal{W}}$, the time to generate a wrapper is T_g, the time of synthesis is T_s, and the time for extracting information from a webpage with a wrapper is T_e. The total time for extracting all information from all websites with AUTOSCRAPER is

$$T_1 = T_G + T_E = (n_sT_g + T_s) + N_{\mathcal{W}}T_e \quad (3)$$

Besides, the time for LLMs directly extracting information from a webpage is T_d, and the total time for extracting all information from all websites directly is

$$T_2 = N_{\mathcal{W}}T_d \quad (4)$$

In a real-world scenario, there are many web pages from the same websites to be extracted. Although generating a wrapper takes more time than extracting directly from a single webpage, the extraction efficiency of subsequent web pages would be significantly improved. To explore how many webpages are needed to make AUTOSCRAPER more efficient in web IE, we calculate the threshold of $N_{\mathcal{W}}$. Suppose $T_1 \leq T_2$, we have

$$T_G + T_E = (n_sT_g + T_s) + N_{\mathcal{W}}T_e \leq N_{\mathcal{W}}T_d \quad (5)$$

$$N_{\mathcal{W}} \geq \frac{n_sT_g + T_s}{T_d - T_e} \quad (6)$$

To verify the efficiency advantages of AUTOSCRAPER in large-scale web information extraction scenarios, we conducted tests on the SWDE dataset. Specifically, we randomly selected a website in each of the 10 domains. We repeat 3 times on AUTOSCRAPER and record the average time to estimate $n_sT_g + T_s$ and T_e. At the same time, we record the average time T_d on 10 web pages with LLM extracting directly. We calculate the threshold of $N_{\mathcal{W}}$ following the Equation 6 and show them in Table 6. It can be observed that the threshold of the page numbers is 19.5 on average, which is significantly lower than the average number of web pages per site in SWDE dataset.

6.4 Error Analysis

We perform an analysis by looking at the recorded action sequence of AUTOSCRAPER with GPT-4-Turbo and identify the following common failure modes. We mainly focus on the cases categorized as unexecutable, over-estimate, and else.

Non-generalizability of webpages The target information and corresponding webpage structures exhibit variations across different webpages, leading to a lack of generalizability in AUTOSCRAPER (i.e., the inability to apply the same rules across all webpages in the same website). For instance, for the task *"Please extract the name of the company offering the job"* in the website job-careerbuilder, most webpages contain the company name, but there is one webpage where the company name is *"Not Available"* on another node of DOM tree.

Miss in multi-valued Presented with the task of generating a scraper for extracting *address* in restaurant webpages or *contact phone number* from university websites, the target information is located in multiple locations in the webpage, such as the information bar, title, etc. Although AUTOSCRAPER is capable of generating action sequences to extract portions of information, crafting a comprehensive action sequence that captures all of the information remains a challenge.

7 Conclusion

In this paper, we introduce the scraper generation task and the paradigm that combines LLMs and scrapers to improve the reusability of the current language-agent-based framework. We then propose AUTOSCRAPER , a two-phase framework including progressive generation and synthesis module to generate a more stable and executable action sequence. Our comprehensive experiments demonstrate that AUTOSCRAPER can outperform the state-of-the-art baseline in the scraper generation task.

Acknowledgement

This work was supported by National Natural Science Foundation of China (No. 62102095). The computations in this research were performed using the CFFF platform of Fudan University. The authors would like to express their sincere gratitude to Alibaba (China) Co., Ltd. and Alibaba Holding-Aicheng Technology-Enterprise Intelligence Business Unit for their support.

Limitation

We introduce a paradigm that combines LLMs with scrapers for web scraper generation tasks and propose AUTOSCRAPER to generate an executable action sequence with progressively understanding the HTML documents. Though experimental results show the effectiveness of our framework, there are still some limits to our work.

First, our framework is restricted to the paradigm in the information extraction task for vertical web pages. LLMs with scrapers provide high efficiency in open-world web IE tasks, but can hardly transfer to existing web environments such as Mind2Web (Deng et al., 2023), WebArena (Zhou et al., 2023).

Second, our framework relies on the performance of backbone LLMs. Enhancing LLMs' ability to understand HTML is a very valuable research question, including corpus collection and training strategy. We will research HTML understanding enhancement in future work.

Ethic statement

We hereby declare that all authors of this article are aware of and adhere to the provided ACL Code of Ethics and honour the code of conduct.

Use of Human Annotations Human annotations are only utilized in the early stages of methodological research to assess the feasibility of the proposed solution. All annotators have provided consent for the use of their data for research purposes. We guarantee the security of all annotators throughout the annotation process, and they are justly remunerated according to local standards. Human annotations are not employed during the evaluation of our method.

Risks The datasets used in the paper have been obtained from public sources and anonymized to protect against any offensive information. Though we have taken measures to do so, we cannot guarantee that the datasets do not contain any socially harmful or toxic language.

References

Marah Abdin, Sam Ade Jacobs, Ammar Ahmad Awan, Jyoti Aneja, Ahmed Awadallah, Hany Awadalla, Nguyen Bach, Amit Bahree, Arash Bakhtiari, Jianmin Bao, Harkirat Behl, Alon Benhaim, Misha Bilenko, Johan Bjorck, Sébastien Bubeck, Qin Cai, Martin Cai, Caio César Teodoro Mendes, Weizhu Chen, Vishrav Chaudhary, Dong Chen, Dongdong Chen, Yen-Chun Chen, Yi-Ling Chen, Parul Chopra, Xiyang Dai, Allie Del Giorno, Gustavo de Rosa, Matthew Dixon, Ronen Eldan, Victor Fragoso, Dan Iter, Mei Gao, Min Gao, Jianfeng Gao, Amit Garg, Abhishek Goswami, Suriya Gunasekar, Emman

Haider, Junheng Hao, Russell J. Hewett, Jamie Huynh, Mojan Javaheripi, Xin Jin, Piero Kauffmann, Nikos Karampatziakis, Dongwoo Kim, Mahoud Khademi, Lev Kurilenko, James R. Lee, Yin Tat Lee, Yuanzhi Li, Yunsheng Li, Chen Liang, Lars Liden, Ce Liu, Mengchen Liu, Weishung Liu, Eric Lin, Zeqi Lin, Chong Luo, Piyush Madan, Matt Mazzola, Arindam Mitra, Hardik Modi, Anh Nguyen, Brandon Norick, Barun Patra, Daniel Perez-Becker, Thomas Portet, Reid Pryzant, Heyang Qin, Marko Radmilac, Corby Rosset, Sambudha Roy, Olatunji Ruwase, Olli Saarikivi, Amin Saied, Adil Salim, Michael Santacroce, Shital Shah, Ning Shang, Hiteshi Sharma, Swadheen Shukla, Xia Song, Masahiro Tanaka, Andrea Tupini, Xin Wang, Lijuan Wang, Chunyu Wang, Yu Wang, Rachel Ward, Guanhua Wang, Philipp Witte, Haiping Wu, Michael Wyatt, Bin Xiao, Can Xu, Jiahang Xu, Weijian Xu, Sonali Yadav, Fan Yang, Jianwei Yang, Ziyi Yang, Yifan Yang, Donghan Yu, Lu Yuan, Chengruidong Zhang, Cyril Zhang, Jianwen Zhang, Li Lyna Zhang, Yi Zhang, Yue Zhang, Yunan Zhang, and Xiren Zhou. 2024. Phi-3 technical report: A highly capable language model locally on your phone.

Mirko Bronzi, Valter Crescenzi, Paolo Merialdo, and Paolo Papotti. 2013. Extraction and integration of partially overlapping web sources. *Proc. VLDB Endow.*, 6(10):805–816.

Xinyun Chen, Maxwell Lin, Nathanael Schärli, and Denny Zhou. 2023. Teaching large language models to self-debug.

Nilesh Dalvi, Ravi Kumar, and Mohamed Soliman. 2011. Automatic wrappers for large scale web extraction. *arXiv preprint arXiv:1103.2406*.

Xiang Deng, Yu Gu, Boyuan Zheng, Shijie Chen, Samuel Stevens, Boshi Wang, Huan Sun, and Yu Su. 2023. Mind2web: Towards a generalist agent for the web.

Pankaj Gulhane, Amit Madaan, Rupesh Mehta, Jeyashankher Ramamirtham, Rajeev Rastogi, Sandeep Satpal, Srinivasan H Sengamedu, Ashwin Tengli, and Charu Tiwari. 2011. Web-scale information extraction with vertex. In *2011 IEEE 27th International Conference on Data Engineering*, pages 1209–1220.

Daya Guo, Qihao Zhu, Dejian Yang, Zhenda Xie, Kai Dong, Wentao Zhang, Guanting Chen, Xiao Bi, Y. Wu, Y. K. Li, Fuli Luo, Yingfei Xiong, and Wenfeng Liang. 2024. Deepseek-coder: When the large language model meets programming – the rise of code intelligence.

Izzeddin Gur, Hiroki Furuta, Austin Huang, Mustafa Safdari, Yutaka Matsuo, Douglas Eck, and Aleksandra Faust. 2023. A real-world webagent with planning, long context understanding, and program synthesis.

Qiang Hao, Rui Cai, Yanwei Pang, and Lei Zhang. 2011. From one tree to a forest: a unified solution for structured web data extraction. In *Proceedings of the 34th International ACM SIGIR Conference on Research and Development in Information Retrieval*, SIGIR '11, page 775–784, New York, NY, USA. Association for Computing Machinery.

Albert Q. Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Emma Bou Hanna, Florian Bressand, Gianna Lengyel, Guillaume Bour, Guillaume Lample, Lélio Renard Lavaud, Lucile Saulnier, Marie-Anne Lachaux, Pierre Stock, Sandeep Subramanian, Sophia Yang, Szymon Antoniak, Teven Le Scao, Théophile Gervet, Thibaut Lavril, Thomas Wang, Timothée Lacroix, and William El Sayed. 2024. Mixtral of experts.

Nicholas Kushmerick. 1997. *Wrapper induction for information extraction*. University of Washington.

Junlong Li, Yiheng Xu, Lei Cui, and Furu Wei. 2022. Markuplm: Pre-training of text and markup language for visually rich document understanding. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 6078–6087.

Bill Yuchen Lin, Ying Sheng, Nguyen Vo, and Sandeep Tata. 2020. Freedom: A transferable neural architecture for structured information extraction on web documents. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pages 1092–1102.

Colin Lockard, Xin Luna Dong, Arash Einolghozati, and Prashant Shiralkar. 2018. Ceres: Distantly supervised relation extraction from the semi-structured web. *arXiv preprint arXiv:1804.04635*.

Colin Lockard, Prashant Shiralkar, and Xin Luna Dong. 2019. Openceres: When open information extraction meets the semi-structured web. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 3047–3056.

Kaixin Ma, Hongming Zhang, Hongwei Wang, Xiaoman Pan, and Dong Yu. 2023. Laser: Llm agent with state-space exploration for web navigation.

Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. 2023. Self-refine: Iterative refinement with self-feedback.

Marco Vinciguerra Marco Perini, Lorenzo Padoan. 2024. Scrapegraph-ai. A Python library for scraping leveraging large language models.

Adi Omari, Sharon Shoham, and Eran Yahav. 2017. Synthesis of forgiving data extractors. In *Proceedings of the tenth ACM international conference on web search and data mining*, pages 385–394.

OpenAI. 2022. Chatgpt.

OpenAI. 2023. Gpt-4 technical report.

OpenAI. 2024. Gpt-4o mini: advancing cost-efficient intelligence.

Baptiste Rozière, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Romain Sauvestre, Tal Remez, Jérémy Rapin, Artyom Kozhevnikov, Ivan Evtimov, Joanna Bitton, Manish Bhatt, Cristian Canton Ferrer, Aaron Grattafiori, Wenhan Xiong, Alexandre Défossez, Jade Copet, Faisal Azhar, Hugo Touvron, Louis Martin, Nicolas Usunier, Thomas Scialom, and Gabriel Synnaeve. 2024. Code llama: Open foundation models for code.

Ritesh Sarkhel, Binxuan Huang, Colin Lockard, and Prashant Shiralkar. 2023. Self-training for label-efficient information extraction from semi-structured web-pages. *Proceedings of the VLDB Endowment*, 16(11):3098–3110.

Tianlin Shi, Andrej Karpathy, Linxi (Jim) Fan, Josefa Z. Hernández, and Percy Liang. 2017. World of bits: An open-domain platform for web-based agents. In *International Conference on Machine Learning*.

Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language agents with verbal reinforcement learning.

Paloma Sodhi, S. R. K. Branavan, and Ryan McDonald. 2023. Heap: Hierarchical policies for web actions using llms.

Abishek Sridhar, Robert Lo, Frank F. Xu, Hao Zhu, and Shuyan Zhou. 2023. Hierarchical prompting assists large language model on web navigation.

Theodore R Sumers, Shunyu Yao, Karthik Narasimhan, and Thomas L Griffiths. 2023. Cognitive architectures for language agents. *arXiv preprint arXiv:2309.02427*.

Gemini Team, Rohan Anil, Sebastian Borgeaud, Yonghui Wu, Jean-Baptiste Alayrac, Jiahui Yu, Radu Soricut, Johan Schalkwyk, Andrew M. Dai, and Anja Hauth. 2023. Gemini: A family of highly capable multimodal models.

Tsaone Swaabow Thapelo, Molaletsa Namoshe, Oduetse Matsebe, Tshiamo Motshegwa, and Mary-Jane Morongwa Bopape. 2021. Sasscal websapi: A web scraping application programming interface to support access to sasscal's weather data. *Data Science Journal*, 20:24–24.

Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. 2023. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*.

Qifan Wang, Yi Fang, Anirudh Ravula, Fuli Feng, Xiaojun Quan, and Dongfang Liu. 2022. Webformer: The web-page transformer for structure information extraction. In *Proceedings of the ACM Web Conference 2022*, pages 3124–3133.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. 2023. Chain-of-thought prompting elicits reasoning in large language models.

Chenxi Whitehouse, Clara Vania, Alham Fikri Aji, Christos Christodoulopoulos, and Andrea Pierleoni. 2023. Webie: Faithful and robust information extraction on the web. *arXiv preprint arXiv:2305.14293*.

Shunyu Yao, Howard Chen, John Yang, and Karthik Narasimhan. 2023. Webshop: Towards scalable real-world web interaction with grounded language agents.

Longtao Zheng, Rundong Wang, Xinrun Wang, and Bo An. 2024. Synapse: Trajectory-as-exemplar prompting with memory for computer control.

Xiaolin Zheng, Tao Zhou, Zukun Yu, and Deren Chen. 2008. Url rule based focused crawler. In *2008 IEEE International Conference on e-Business Engineering*, pages 147–154. IEEE.

Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. 2023. Webarena: A realistic web environment for building autonomous agents.

Yichao Zhou, Ying Sheng, Nguyen Vo, Nick Edmonds, and Sandeep Tata. 2021. Simplified dom trees for transferable attribute extraction from the web. *arXiv preprint arXiv:2101.02415*.

A Experiments

A.1 Main results on EXTENDED SWDE

Because EXTENDED SWDE dataset focuses on *OpenIE* task (the relation is also expected to be extracted), we first map relations into a predefined list of attributes and remove unusual ones. Specifically, we conducted experiments with 294 attributes from 21 websites selected from the EXTENDED SWDE dataset.

Table 7 shows the result. By comparing Table 2, we find that: 1) Under complex extraction task settings (multiple target values and ambiguous problem description), the closed-source LLMs perform better in generating executable action sequences compared to the open-source LLMs. 2) There are some tasks with unclear descriptions, such as the *"Calendar System"* and *"Facilities and Programs Offered"* on university websites, which affect the wrapper generation performance of all methods.

A.2 Main results on DS1

Due to DS1 only contains 166 hand-crafted webpages, and for each website, there are only two webpages, so we take one webpage for inference and the other for evaluation. Meanwhile, due to the number of seed websites equal to one, we test three methods without applying the synthesis module described in Section 4.3.

Table 8 shows the result in the DS1 dataset. Among all LLMs with three methods, GPT-4-Turbo + AUTOSCRAPER achieves the best performance, and AUTOSCRAPER beats the other two methods in all LLMs, which is consistent with our conclusion.

A.3 Generate with Golden Label

To better illustrate the effectiveness of our framework in generating executable action sequences, we compare the performance of COT, Reflexion, and AUTOSCRAPER , while answering the instruction. By offering the same extraction targets, we can effectively detect the performance of different frameworks in generating action sequences.

Table 9 shows experimental results, from which we can have the following observations: 1) Our proposed progressive understanding framework still effectively enhances the model's performance under this setting; 2) LLMs still suffer in accurately understanding web page contents with semi-structured markup languages, which illustrate the performance gap between Table 2 and Table 9;

Algorithm 1: Algorithm for progressive understanding

Data: origin HTML code h_0, task instruction I, max retry times d_{max}
Result: Executable action sequence $\mathcal{A}_{seq}$ to extract the value in the HTML

Initial history $\mathcal{A}_{seq} \leftarrow [], k = 0$;
while *True* **do**
  if $k > d_{max}$ **then** break;
  // Top-down
  $value, xpath \leftarrow \textbf{LLM}_g(h_k, I)$;
  $result \leftarrow \textbf{Parser}_{text}(h_k, xpath)$;
  if *result* == *value* **then** break;
  // Step-back
  repeat
    $xpath \leftarrow xpath + "/.."$;
    $h_{k+1} \leftarrow \textbf{Parser}_{node}(h_k, xpath)$;
  until *h contains value*;
  Append($\mathcal{A}_{seq}, xpath$);
  $k \leftarrow k + 1$;
end
return $\mathcal{A}_{seq}$

3) Compared to closed-source LLMs, even provided with golden labels, Open-source LLMs are unable to achieve sustained performance improvement. This phenomenon demonstrates that the bottleneck for these models lies not in understanding the webpage content but in understanding the webpage's hierarchical structure itself.

B Analysis on AUTOSCRAPER

B.1 Comparison with COT & Reflexion

Figure 4 more intuitively shows the specific differences between different baselines in the experiment. The most significant difference between AUTOSCRAPER and other methods lies in whether the hierarchical structure of web pages is utilized to help LLMs reduce the difficulty of complex web structures. COT only executes one turn while the other executes multiple turns and can learn from the failed execution of the wrapper. Compared to the Reflexion method, AUTOSCRAPER employs top-down and step-back operations to prune the DOM tree during each XPath generation process, thereby reducing the length of the web page. In contrast, the Reflexion method can only reflect and regenerate after producing an unexecutable XPath, which does not effectively simplify the webpage.

Models	Method	Executable Evaluation						IE Evaluation		
		Correct(↑)	Prec	Reca	Unex.(↓)	Over.	Else	Prec	Reca	F1
Closed-source LLMs										
GPT-3.5-Turbo	COT	35.19	3.48	4.53	55.40	0.35	1.05	88.66	42.86	41.28
	Reflexion	43.90	1.74	2.09	49.13	0.35	2.79	**93.46**	49.58	48.66
	AutoScraper	**46.34**	4.18	8.01	**34.84**	0.35	6.27	84.65	**61.88**	**57.74**
Gemini Pro	COT	34.49	2.09	6.62	49.13	0.35	7.32	81.09	46.55	42.40
	Reflexion	34.15	2.09	6.97	51.57	0.35	4.88	**84.43**	45.19	41.66
	AutoScraper	**35.89**	5.23	10.10	**42.86**	0.35	5.57	83.80	**52.83**	**47.80**
GPT-4-o-mini	COT	45.79	4.38	4.71	38.72	0.00	0.64	88.59	57.97	56.32
	Reflexion	39.06	7.07	2.02	47.47	0.34	4.04	**95.03**	49.07	48.66
	AutoScraper	**56.23**	5.39	5.05	**27.27**	0.00	6.06	91.12	**69.45**	**67.56**
GPT-4-Turbo	COT	56.10	2.44	7.32	29.27	0.35	4.53	85.15	68.35	65.08
	Reflexion	**64.81**	4.18	5.57	19.51	0.35	5.57	**87.39**	77.81	75.85
	AutoScraper	64.11	3.48	6.27	**15.33**	0.35	10.45	82.71	**80.25**	**76.21**
Open-source LLMs										
Phi-3-medium	COT	11.78	1.01	5.05	79.46	0.34	2.36	91.03	19.08	16.28
	Reflexion	12.66	1.90	1.90	82.28	0.00	1.27	**93.87**	16.03	15.42
	AutoScraper	**21.15**	2.88	7.69	**64.42**	0.00	3.85	87.88	**33.39**	**30.29**
CodeLlama	COT	9.01	1.29	2.15	85.84	0.00	1.72	87.22	12.62	11.21
	Reflexion	**13.73**	1.72	3.00	**80.26**	0.00	1.29	89.41	**17.76**	**16.01**
	AutoScraper	11.16	0.00	1.72	85.84	0.00	1.29	**92.49**	13.29	12.52
Mixtral 8×7B	COT	32.40	1.05	4.88	57.14	0.35	4.18	**87.87**	41.20	38.30
	Reflexion	29.62	1.05	4.18	62.02	0.35	2.79	83.44	36.44	33.64
	AutoScraper	**40.77**	3.83	9.76	**38.33**	0.35	6.97	82.50	**58.14**	**52.50**
Deepseek-coder	COT	**38.33**	3.83	6.62	**47.74**	0.35	3.14	81.32	**48.52**	**44.80**
	Reflexion	36.24	3.48	3.83	51.92	0.00	4.53	83.53	45.03	43.64
	AutoScraper	37.63	2.44	5.92	50.52	0.35	3.14	**86.91**	47.09	44.33

Table 7: The executable evaluation and IE evaluation of LLMs with three frameworks in Extended Swde dataset. We examine 6 LLMs, including 3 closed-source LLMs and 3 open-source LLMs.

B.2 Further Study with AutoScraper

The length of the action sequence is dependent on the LLM capability. To comprehensively explore the performance of different LLMs in understanding web page structure, we explore the impact of models on the number distribution of the steps. In particular, we collect all the action sequences and calculate the average steps of AutoScraper with different LLMs. The experimental result is reported in Table 10, 11 and 12.

We observe that AutoScraper with stronger LLMs generates fewer lengths of action sequence. AutoScraper with GPT-4-Turbo generates 1.57 steps on average, while the AutoScraper with Phi-3-medium generates 3.62 steps on average. This phenomenon can be interpreted as more powerful models having a better understanding of the web page hierarchical structure, thus being able to accurately output the appropriate XPaths in longer/deeper web pages, thereby reducing the number of steps.

XPath fragility within AutoScraper The fragility of XPath often refers to the characteristic of XPath expressions becoming ineffective or inaccurately matching the target element when faced with new web pages. This is mainly due to XPath specifying specific information through *predicates*, such as text, `@class`, etc.

We mainly focus on the fragility of text because these webpages are from the same websites (i.e. `@class` is a good characteristic for generating stable action sequences). Table 14 shows XPath expressions that rely on text. We aim to explore the reusability of generating XPath based on text features. We manually calculated the proportion of bad cases with two types of predicates, *contains* and *equal* [6]. The results in Table 13 show that the stronger LLMs capability, the lower the proportion of bad cases with AutoScraper . However, it should be noted that the current SoTA LLM GPT-4-Turbo still suffers from an XPath fragility problem, which indicates that relying entirely on LLMs to generate reliable XPath still has some distance to go.

C Dataset Statistic

Table 15, 16, 17 shows the detailed statistic about the semi-structure web information extraction dataset Swde, Extended Swde and Ds1.

[6] https://www.w3schools.com/xml/xpath_syntax.asp

Models	Method	Executable Evaluation						IE Evaluation		
		Correct(↑)	Prec	Reca	Unex.(↓)	Over.	Else	Prec	Reca	F1
Closed-source LLMs										
GPT-3.5-Turbo	COT	32.65	4.08	8.16	53.06	0.00	2.04	90.56	43.54	41.16
	Reflexion	36.73	8.16	4.08	51.02	0.00	0.00	**95.56**	44.22	43.75
	AutoScraper	**48.98**	4.08	0.00	**44.90**	0.00	2.04	94.90	**51.70**	**52.38**
Gemini Pro	COT	17.72	2.53	3.80	75.95	0.00	0.00	90.82	22.88	22.10
	Reflexion	20.25	10.13	1.27	65.82	0.00	2.53	88.83	26.93	27.66
	AutoScraper	**43.04**	15.19	3.80	**34.18**	0.00	3.80	**93.76**	**55.97**	**56.92**
GPT-4-o-mini	COT	46.99	3.61	4.82	42.17	0.00	2.41	79.74	55.34	53.77
	Reflexion	38.55	13.25	2.41	45.78	0.00	0.00	**91.40**	43.68	43.86
	AutoScraper	**53.01**	6.02	4.82	**34.94**	0.00	1.20	79.06	**61.03**	**60.10**
GPT-4-Turbo	COT	50.60	9.64	6.02	30.12	0.00	3.61	93.60	65.75	64.73
	Reflexion	50.60	10.84	4.82	33.73	0.00	0.00	**96.85**	62.65	63.50
	AutoScraper	**57.83**	15.66	4.82	**16.87**	0.00	4.82	92.88	**74.95**	**75.52**
Open-source LLMs										
Phi-3-medium	COT	9.64	4.82	0.00	85.54	0.00	0.00	95.18	11.76	12.28
	Reflexion	7.23	0.00	1.20	90.36	0.00	1.20	**97.87**	9.47	8.89
	AutoScraper	**22.89**	3.61	3.61	**69.88**	0.00	0.00	88.00	**28.22**	**26.60**
CodeLlama	COT	2.70	2.70	5.41	89.19	0.00	0.00	78.72	10.62	9.19
	Reflexion	8.82	0.00	5.88	85.29	0.00	0.00	**94.12**	14.41	12.69
	AutoScraper	**13.51**	0.00	5.41	**81.08**	0.00	0.00	84.12	**18.92**	**17.39**
Mixtral 8×7B	COT	17.72	6.33	0.00	74.68	0.00	1.27	**94.81**	21.15	22.01
	Reflexion	22.78	6.33	1.27	69.62	0.00	0.00	94.15	28.03	28.20
	AutoScraper	**36.71**	11.39	6.33	**43.04**	0.00	2.53	91.59	**48.52**	**48.23**
Deepseek-coder	COT	25.30	9.64	2.41	60.24	0.00	2.41	**92.47**	34.71	35.65
	Reflexion	22.89	6.02	3.61	65.06	0.00	2.41	90.21	31.43	32.04
	AutoScraper	**39.76**	10.84	6.02	**42.17**	0.00	1.20	90.43	**51.39**	**50.28**

Table 8: The executable evaluation and IE evaluation of LLMs with three frameworks in DS1 dataset. We examine 8 LLMs, including 4 closed-source LLMs and 4 open-source LLMs.

Models	Method	Executable Evaluation					
		Correct(↑)	Prec	Reca	Unex.(↓)	Over.	Else
Closed-source LLMs							
GPT-3.5-Turbo	COT	41.70	12.92	7.38	35.42	0.74	1.85
	Reflexion	47.23	16.24	2.21	33.21	0.37	0.74
	AutoScraper	**56.89**	19.43	5.65	**13.43**	0.71	3.89
Gemini Pro	COT	33.44	9.38	9.06	44.69	0.94	2.50
	Reflexion	35.31	9.38	6.88	43.75	1.56	3.12
	AutoScraper	**45.31**	13.44	6.25	**30.31**	1.25	3.44
GPT-4-o-mini	COT	56.59	12.54	8.04	17.36	0.96	0.45
	Reflexion	62.38	10.29	1.61	23.15	0.64	1.93
	AutoScraper	**67.20**	12.22	3.86	**12.22**	0.96	3.54
GPT-4-Turbo	COT	61.88	11.56	9.06	11.56	1.25	4.69
	Reflexion	71.25	7.19	4.69	14.37	0.94	1.56
	AutoScraper	**75.31**	10.94	4.37	**4.06**	0.63	4.69
Open-source LLMs							
Phi-3-medium	COT	11.11	4.13	1.27	82.22	0.00	1.27
	Reflexion	12.19	5.27	7.59	72.43	0.31	2.21
	AutoScraper	**27.27**	16.45	9.52	**41.56**	0.87	4.33
CodeLlama	COT	21.40	6.27	2.21	66.79	0.74	2.58
	Reflexion	22.21	4.93	3.94	66.95	0.49	1.48
	AutoScraper	**26.20**	12.55	5.54	**53.51**	0.00	2.21
Mixtral 8×7B	COT	27.50	7.50	5.31	56.87	0.94	1.87
	Reflexion	34.69	8.13	5.31	49.06	0.63	2.19
	AutoScraper	**45.62**	11.56	5.94	**32.50**	1.25	3.12
Deepseek-coder	COT	35.00	18.75	5.31	36.25	0.63	4.06
	Reflexion	**38.75**	11.87	2.81	42.19	0.63	3.75
	AutoScraper	38.44	20.94	4.06	**31.56**	0.94	6.56

Table 9: The executable and IE evaluation with 8 LLMs on SWDE dataset with golden label.

Models	1	2	3	4	5	Avg.
GPT-4-Turbo	214	61	13	18	10	1.57
GPT-4-o-mini	183	35	20	10	60	2.12
GPT-3.5-Turbo	124	61	38	22	73	2.56
Gemini Pro	94	52	33	27	105	2.99
Mixtral 8×7B	89	53	43	24	104	3.00
Phi-3-medium	47	52	28	26	155	3.62
Deepseek-coder	137	70	55	29	23	2.14
CodeLlama	75	35	32	18	80	2.97

Table 10: Length of action sequence of AutoScraper based on different LLMs in SWDE dataset.

Models	1	2	3	4	5	Avg.
GPT-4-Turbo	28	23	15	11	5	2.29
GPT-4-o-mini	50	10	5	1	16	2.06
GPT-3.5-Turbo	15	10	3	5	7	2.48
Gemini Pro	22	17	13	7	20	2.82
Mixtral 8×7B	16	13	7	11	29	3.32
Phi-3-medium	14	15	6	2	46	3.61
Deepseek-coder	34	20	17	10	2	2.11
CodeLlama	18	6	6	9	33	3.46

Table 11: Length of action sequence of AutoScraper based on different LLMs in DS1 dataset.

D Prompt List

D.1 Task Prompt

Table 18 shows the task prompt we design for each attribute for SWDE.

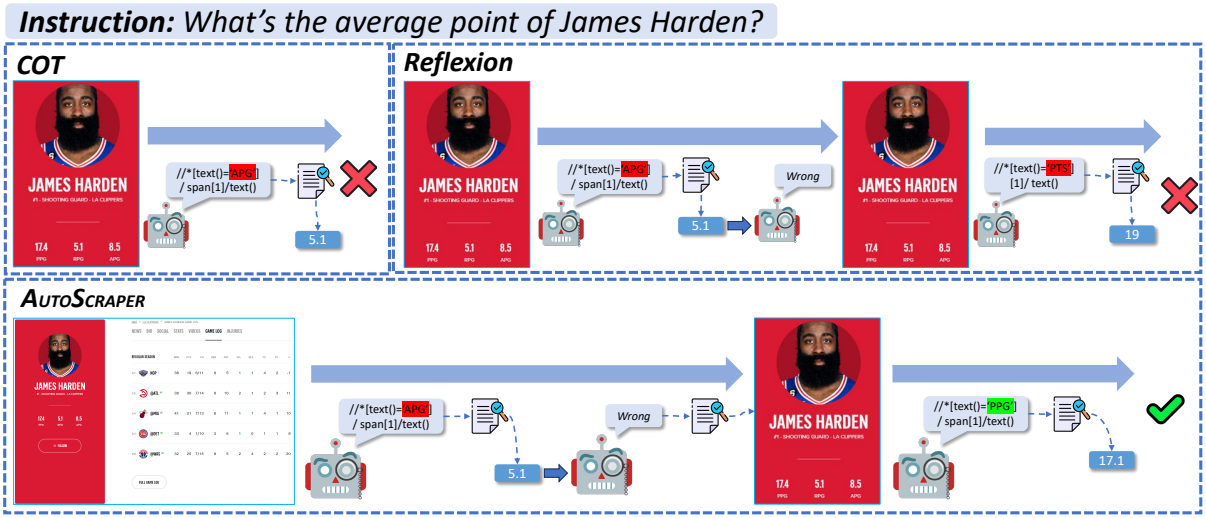


Figure 4: Comparison of AUTOSCRAPER with COT and Reflexion.

Models	1	2	3	4	5	Avg.
GPT-4-Turbo	61	40	45	53	76	3.15
GPT-4-o-mini	133	31	17	15	91	2.65
GPT-3.5-Turbo	88	35	48	23	97	3.02
Gemini Pro	60	41	29	28	132	3.45
Mixtral 8×7B	51	38	26	29	138	3.59
Phi-3-medium	43	39	34	25	144	3.66
Deepseek-coder	120	79	35	33	20	2.14
CodeLlama	53	31	6	6	14	2.06

Table 12: Length of action sequence of AUTOSCRAPER based on different LLMs in EXTENDED SWDE dataset.

Models	Contains	Equal(=)
GPT4	0.61%	2.90%
GPT-3.5-Turbo	9.33%	9.78%
Gemini Pro	10.62%	14.29%
Mixtral 8×7B	12.88%	8.55%
Deepseek-Coder	11.63%	7.55%
CodeLlama	18.75%	14.29%
Mistral 7B	18.18%	33.33%

Table 13: Bad case ratio in two types of predicate.

D.2 Module Prompt

We provide a comprehensive list of all the prompts that have been used in this study, offering a clear reference to understand our experimental approach.

	Good case	Bad case
Question	*Here's a webpage on detail information with detail information of an NBA player. Please extract the* ***height*** *of the player.*	*Here's a webpage with detailed information about a university. Please extract the* ***contact phone number*** *of the university.*
Case	*//div[@class='gray200B-dyContent']/b[* ***contains(text(),'Height:')*** *]/following-sibling::text()*	*//div[@class='infopage']//h5[* ***contains (text(), '703-528-7809')*** *]*

Table 14: Examples of XPath fragility. The green focuses on the common information across different webpages, while the red focuses on specific information of seed webpages.

Domain	Attribute	Website	Num	Domain	Attribute	Website	Num
Auto	model price engine fuel_economy	aol	2000	**Movie**	ttitle director genre mpaa_rating	allmovie	2000
		autobytel	2000			amctv	2000
		automotive	1999			boxofficemojo	2000
		autoweb	2000			hollywood	2000
		carquotes	2000			iheartmovies	2000
		cars	657			imdb	2000
		kbb	2000			metacritic	2000
		motortrend	1267			msn	2000
		msn	2000			rottentomatoes	2000
		yahoo	2000			yahoo	2000
Book	title author isbn_13 publisher pub_date	abebooks	2000	**NBAPlayer**	name team height weight	espn	434
		amazon	2000			fanhouse	446
		barnesandnoble	2000			foxsports	425
		bookdepository	2000			msnca	434
		booksamillion	2000			nba	434
		bookorders	2000			si	515
		buy	2000			slam	423
		christianbook	2000			usatoday	436
		deepdiscount	2000			wiki	420
		waterstone	2000			yahoo	438
Camera	model price manufacturer	amazon	1767	**Restaurant**	name address phone cuisine	fodors	2000
		beachaudio	247			frommers	2000
		buy	500			zagat	2000
		compsource	430			gayot	2000
		ecost	923			opentable	2000
		jr	367			pickaretaurant	2000
		newegg	220			restaurantica	2000
		onsale	261			tripadvisor	2000
		pcnation	234			urbanspoon	2000
		thenerd	309			usdiners	2000
Job	title company location date_posted	careerbuilder	2000	**University**	name phone website type	collegeboard	2000
		dice	2000			collegenavigator	2000
		hotjobs	2000			collegeprowler	2000
		job	2000			collegetoolkit	2000
		jobcircle	2000			ecampustours	1063
		jobtarget	2000			embark	2000
		monster	2000			matchcollege	2000
		nettemps	2000			princetonreview	615
		rightitjobs	2000			studentaid	2000
		techcentric	2000			usnews	1027

Table 15: Detail statistic of SWDE dataset.

Prompt of Top-down Operation

```
Here's the HTML extraction task:
Task description: Please read the following HTML code, and then return an Xpath
    that can recognize the element in the HTML matching the instruction below.
Instruction: {0}
We will offer some history about the thought and the extraction result. Please
    reflect on the history trajectory and adjust the xpath rule for better and
    more exact extraction. Here are some hints:
1. Judge whether the results in the history are consistent with the expected
    value. Please pay attention to the following case:
   1) Whether the extraction result contains some irrelevant elements
   2) Whether the scraper returns an empty result
   3) The raw values containing redundant separators are considered consistent
       because we will postprocess them.
2. Re-thinking the expected value and how to find it depends on the xpath code
3. Generate a new or keep the origin xpath depending on the judgement and
    thinking following the hints:
   1. Do not output the xpath with the exact value or element that appears in the
       HTML.
   2. Do not output the xpath that indicates multiple nodes with different values
       . It would be appreciated to use more @class and [num] to identify the
       different nodes that may share the same xpath expression.
   3. If the HTML code doesn't contain suitable information to match the
       instruction, keep the xpath attribute blank.

Please output in the following JSON format:
{
   "thought": "", # thought of why the xpaths in history do not work and how to
       adjust the xpath
   "consistent": "", # whether the extracted result is consistent with the
       expected value, return yes/no directly
   "value": "", # the value extracted from the HTML that matches the task
       description
   "xpath": "", # a new XPath that is different from the XPath in the following
       history if not consistent
}

And here's the history of the thought, xpath and result extracted by scraper.
{1}

Here's the HTML code:
```
{2}
```
```

Prompt of Step-back Operation

```
Your main task is to judge whether the following HTML code contains all the
    expected values, which are recognized beforehand.
Instruction: {0}
And here's the value: {1}
The HTML code is as follows:
```
{2}
```
```

Please output your judgement in the following JSON format:
{
   "thought": "", # a brief thinking about whether the HTML code contains
       expected value
   "judgement": "" # whether the HTML code contains all extracted value. Return
       yes/no directly.
}
```
```

Prompt of Synthesis

```
You're a perfect discriminator who is good at HTML understanding as well.
    Following the instructions, there are some action sequences written from
    several HTML and the corresponding results extracted from several HTML.
    Please choose one that can be best potentially adapted to the same extraction
     task on other web pages on the same websites. Here are the instructions for
    the task:
Instructions: {0}
The action sequences and the corresponding extracted results with different
    sequences on different webpage are as follows:
{1}

Please output the best action sequence in the following JSON format:
{
   "thought": "" # brief thinking about which to choose
   "number": " # the best action sequence chosen from the candidates, starts from
       0.  If there is none, output 0.
}
```

Domain	Website	# Attributes
Movie	allmovie	20
	amctv	13
	hollywood	12
	iheartmovies	8
	imdb	34
	metacritic	17
	rottentomatoes	10
	yahoo	10
NBAPlayer	espn	10
	fanhouse	14
	foxsports	10
	msnca	12
	si	12
	slam	12
	usatoday	5
	yahoo	9
University	collegeprowler	18
	ecampustours	14
	embark	23
	matchcollege	15
	usnews	19

Table 16: Detail statistic of EXTEND SWDE dataset.

Domain	Attribute	Website
Book	title author price	abebooks alibris barnesandnoble fishpond infibeam powells thriftbooks
E-commerce	title price	amazoncouk bestbuy dabs ebay pcworld tesco uttings
Hotel	address price title	agoda expedia hotels hoteltravel javago kayak ratestogo venere
Movie	actor genre title	123movieto hollywoodreporter imdb mediastinger metacritic rottentomatoes themoviedb yidio

Table 17: Detail statistic of DS1 dataset.

Domain	Task prompt	Prompt
Auto	Here's a webpage with detailed information about an auto.	Please extract the model of the auto. Please extract the price of the auto. Please extract the engine of the auto. Please extract the fuel efficiency of the auto.
Book	Here's a webpage with detailed information about a book.	Please extract the title of the book. Please extract the author of the book. Please extract the isbn number of the book. Please extract the publisher of the book. Please extract the publication date of the book.
Camera	Here's a webpage with detail information of camera.	Please extract the product name of the camera. Please extract the sale price of the camera. Please extract the manufacturer of the camera.
Job	Here's a webpage with detailed information about a job.	Please extract the title of the job. Please extract the name of the company that offers the job. Please extract the working location of the job. Please extract the date that post the job.
Movie	Here's a webpage with detailed information about a movie.	Please extract the title of the movie. Please extract the director of the movie. Please extract the genre of the movie. Please extract the MPAA rating of the movie.
NBAPlayer	Here's a webpage with detailed information about an NBA player.	Please extract the name of the player. Please extract the team of the player he plays now. Please extract the height of the player. Please extract the weight of the player.
Restaurant	Here's a webpage with detailed information about a restaurant.	Please extract the restaurant's name. Please extract the restaurant's address. Please extract the restaurant's phone number. Please extract the cuisine that the restaurant offers.
University	Here's a webpage on detailed information about a university.	Please extract the name of the university. Please extract the contact phone number of the university. Please extract the website url of the university. Please extract the type of the university.

Table 18: Prompts for scraper generation task in SWDE dataset.